

Summary of definitions, formulas, invariants, operations, and complexity results covered in the course. Repository: <https://github.com/byepesg/DS-262>

1. Asymptotic Analysis

Describes how computational cost grows with input size n .

BIG-O (UPPER BOUND)

$$f(n) \in O(g(n)) \iff \exists c > 0, n_0 : 0 \leq f(n) \leq c g(n), \forall n \geq n_0$$

BIG-Ω (LOWER BOUND)

$$f(n) \in \Omega(g(n)) \iff \exists c > 0, n_0 : 0 \leq c g(n) \leq f(n), \forall n \geq n_0$$

BIG-Θ (TIGHT BOUND)

$$f(n) \in \Theta(g(n)) \iff \exists c_1, c_2 > 0, n_0 : c_1 g(n) \leq f(n) \leq c_2 g(n)$$

$$\Theta(g(n)) = O(g(n)) \cap \Omega(g(n))$$

2. Common Growth Rates

$$1 \prec \log n \prec n \prec n \log n \prec n^2 \prec n^3 \prec 2^n \prec n!$$

Complexity	Name
$O(1)$	Constant
$O(\log n)$	Logarithmic
$O(n)$	Linear
$O(n \log n)$	Linearithmic
$O(n^2)$	Quadratic
$O(n^3)$	Cubic
$O(2^n)$	Exponential
$O(n!)$	Factorial

3. Logarithm Identities

$$\log_a n = \frac{\log_b n}{\log_b a} \Rightarrow \log_a n = \Theta(\log_b n)$$

Base is irrelevant asymptotically: $O(\log_2 n) = O(\log_{10} n) = O(\ln n)$.

$$\log(ab) = \log a + \log b \quad \log\left(\frac{a}{b}\right) = \log a - \log b \quad \log(a^k) = k \log a$$

4. Useful Summations

$$\sum_{i=1}^n 1 = n = \Theta(n)$$

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} = \Theta(n^2)$$

$$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6} = \Theta(n^3)$$

$$\sum_{i=0}^n r^i = \frac{r^{n+1} - 1}{r - 1}, \quad r \neq 1$$

$$\text{If } 0 < r < 1: \sum_{i=0}^{\infty} r^i = \frac{1}{1-r}$$

5. Cost Functions & Loops

Simple loop $T(n) = an + b \in \Theta(n)$

Nested loop $n \cdot n = n^2 \in \Theta(n^2)$

Triangular loop $\sum_{i=0}^{n-1} (i+1) = \frac{n(n+1)}{2} \in \Theta(n^2)$

6. Arrays

Elements of equal size in contiguous memory.

$$\text{addr}(A[i]) = B + i \cdot s$$

B = base address, s = element size $\Rightarrow$ random access is $\Theta(1)$.

Operation	Cost
Access by index	$O(1)$
Update by index	$O(1)$
Search (unsorted)	$O(n)$
Insert/Delete at front	$O(n)$
Insert/Delete arbitrary	$O(n)$

Key idea: arrays give efficient random access because the element address is computed directly from its index.

7. Singly Linked Lists

Node = (key, next)

head $\rightarrow v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_n \rightarrow \text{null}$

head = null $\iff$ list is empty

PUSH FRONT – $O(1)$

newNode.next = head; head = newNode

PUSH BACK

Without tail: $O(n)$ (must traverse). With tail reference: $O(1)$.

Operation	Cost
pushFront / topFront / popFront	$O(1)$
Search / Access by position	$O(n)$
pushBack (no tail)	$O(n)$
pushBack (with tail)	$O(1)$

8. Stacks

LIFO: Last In, First Out

push(x), pop(), top(), isEmpty()

top(push(S,x)) = x pop(push(S,x)) = S

ARRAY REPRESENTATION

Empty when top = -1. Insert: stack[++top]=x. Remove: stack[top--].

Operation	Cost
push / pop / top / isEmpty	$O(1)$

The Stack ADT does not provide arbitrary access – behavior is defined solely through LIFO operations.

9. Queues

FIFO: First In, First Out

enqueue(x), dequeue(), front(), isEmpty()

front $\rightarrow v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_n \leftarrow \text{rear}$

Operation	Cost
enqueue / dequeue / front / isEmpty	$O(1)$

Stack $\Rightarrow$ LIFO. Queue $\Rightarrow$ FIFO.

10. Circular Arrays

For capacity m :

$$\text{next}(i) = (i + 1) \bmod m \quad \text{prev}(i) = (i - 1 + m) \bmod m$$

The $+m$ guarantees a non-negative value before the modulo.

Circular queue update: $\text{rear} = (\text{rear} + 1) \bmod m$, $\text{front} = (\text{front} + 1) \bmod m$.

$$0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow m - 1 \rightarrow 0$$

Circular arrays reuse freed positions after **dequeue**, avoiding shifting all elements.

11. Dynamic Arrays

n = stored elements, C = capacity.

$$0 \leq n \leq C$$

When $n = C$, resize is required. Common strategy: $C_{\text{new}} = 2C$, then copy elements $\Rightarrow$ cost $\Theta(n)$. Resizing does *not* happen every insertion.

12. Amortized Analysis

Studies the cost of a **sequence** of operations, not a single one.

Doubling strategy: copies happen near capacities 1, 2, 4, 8, ...

$$1 + 2 + 4 + 8 + \dots < 2n$$

Total cost for n insertions: $T(n) = O(n) \Rightarrow$ amortized cost $\frac{T(n)}{n} = O(1)$.

Dynamic array append = $O(1)$ amortized

Worst case (one append): $O(n)$. **Amortized (sequence):** $O(1)$.

13. Quick Complexity Table

Structure	Acc.	Ins.Fr	Ins.Bk
Array	$O(1)$	$O(n)$	$O(1)^*$
Dynamic Array	$O(1)$	$O(n)$	$O(1)^\dagger$
Linked List	$O(n)$	$O(1)$	$O(n)$
Linked List+tail	$O(n)$	$O(1)$	$O(1)$
Stack	–	$O(1)$	–
Queue	–	–	$O(1)$

* Assuming available capacity. † Amortized, geometric resizing.

14. ADT vs. Representation

An **ADT** specifies *behavior*; an **implementation** determines how it is represented in memory.

ADT	Possible implementations
Stack	Array, Linked List
Queue	Linked List, Circular Array

ADT $\neq$ Implementation

The ADT determines **what** operations mean; the representation determines **how** they are implemented and how much they cost.

15. Choosing a Data Structure

Need	Structure
Fast random access by index	Array / Dynamic Array
Frequent insertion at front	Linked List
Last in leaves first	Stack
First in leaves first	Queue
Array FIFO, no shifting	Circular Array
Auto-growing array	Dynamic Array

16. Core Ideas

- Asymptotic analysis describes cost growth with input size.
- O , Ω , Θ describe different asymptotic bounds.
- Memory representation determines operation cost.
- Arrays give $\Theta(1)$ access via direct address computation.
- Linked lists trade contiguous memory for node references.
- Stacks/Queues are ADTs – semantics independent of representation.
- Modular arithmetic makes an array behave circularly.
- Dynamic arrays trade rare resizes for $O(1)$ amortized append.
- Complexity should target the operations that matter for the problem.

★. Coming Next

- Tree terminology
- Tree traversals
- Binary Search Trees
- AVL Trees
- Heaps
- Heap-Sort
- Disjoint Sets
- Hash Functions
- Hash Tables
- Rabin–Karp
- Graph representations
- DFS / BFS
- A*